THREAT MODELING WEB APPLICATION

Project Report & Step-by-Step GitHub Guide

Objective	Identify application security risks systematically and recommend proportionate defensive controls.
Deliverables	Threat-model overview, STRIDE threat register, risk ranking, and prioritized hardening checklist.
Technology	HTML, CSS, JavaScript, GitHub, and GitHub Pages (optional hosting).

Prepared as a beginner-friendly educational project document.

1. Project Overview

Threat modeling is a structured way to understand what a system needs to protect, how it might be misused, and which safeguards should be implemented first. This project is a small educational web application that helps a student record a threat, assign likelihood and impact scores, calculate a risk score, and review recommended defensive actions.

Problem Statement

Application teams can overlook security risks when assets, trust boundaries, external services, and potential abuse cases are not documented. A simple threat register makes these risks visible and helps prioritize remediation.

Project Objectives

• Identify important assets such as user accounts, application data, APIs, and external services.

• List plausible threats using STRIDE or a comparable method.

• Rank risks using likelihood and impact.

• Recommend proportionate defensive controls and track checklist completion.

• Publish project files and supporting documentation in a GitHub repository.

2. Scope and Technologies

Technology	Purpose
HTML	Page structure, form fields, and checklist
CSS	Layout, colours, spacing, and responsive presentation
JavaScript	Risk-score calculation and dynamic result display
GitHub	Version control and project submission repository
GitHub Pages	Optional static hosting for a public demo

Scope limitation

This starter project is a documentation and risk-scoring aid. It does not scan systems, verify vulnerabilities, or replace a professional security review. Use fictional examples; do not enter real passwords, secrets, or sensitive data.

3. System Model: Users, Data, Trust Boundaries

Actors: Student / analyst (records threats); application visitor (views the static page).
Data/assets: asset list, threat descriptions, likelihood and impact values, risk scores, checklist status.
External dependency: GitHub repository and optionally GitHub Pages.
Trust boundaries: browser input fields; browser-to-hosted static files; repository write permissions. The starter app processes scores in the browser and does not send form entries to a server.

Student / Analyst	→ enters threat details →	**Browser-based app**
Repository maintainer	→ commits project files →	**GitHub repository / Pages**

Textual flow: Analyst → browser UI → risk score and suggested action. Maintainer → GitHub repository → optional public website.

4. STRIDE Threat Register

STRIDE is a common threat-classification mnemonic: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. The following are illustrative risks for discussion, not findings from a scan.

STRIDE category	Example threat	Defensive control
Spoofing	An account is accessed by someone other than its owner.	Use MFA, secure session handling, and strong authentication.
Tampering	Untrusted input changes data unexpectedly.	Validate server-side input and enforce integrity checks.
Repudiation	An important action cannot be reliably traced.	Keep protected, time-stamped audit logs.
Information Disclosure	Private data is exposed to an unauthorized user.	Enforce access control; encrypt data in transit and at rest.
Denial of Service	Excessive requests reduce service availability.	Apply rate limits, monitoring, and capacity safeguards.
Elevation of Privilege	A low-privilege user gains higher permissions.	Apply least privilege and test authorization rules.

5. Risk Scoring and Prioritization

Risk score = Likelihood × Impact, with each value rated from 1 to 5. Maximum score: 25.

Score	Level	Suggested response
1–5	Low	Record and monitor; review when the system changes.
6–14	Medium	Assign an owner and plan reasonable mitigation.
15–25	High	Prioritize mitigation and verify the control before release.

Worked example

Threat: unauthorized access to an account. Likelihood = 4 (likely), Impact = 5 (critical). Score = 4 × 5 = 20, so the example is High priority. Suggested actions: enable MFA, review session settings, monitor suspicious sign-ins, and test account recovery controls.

6. Prioritized Hardening Checklist

P1 — Critical foundation: Protect repository access with MFA; review who can write to the repository; never commit API keys or passwords.

P1 — Input and access: Validate all user input; if a server is added later, enforce validation and authorization on the server too.

P1 — Data protection: Minimize stored data; use HTTPS for hosted pages; do not collect real sensitive data in this demo.

P2 — Visibility: Enable platform/account security notifications and keep an incident contact and response steps documented.

P2 — Recovery: Keep a clean copy of source code and document how to restore the project.

P3 — Review: Reassess threats after changes; track each risk owner, mitigation, and residual risk.

7. Step-by-Step GitHub Setup

1. **Create repository:** Sign in at github.com, choose New repository, name it `threat-modeling-app`, and create it.

2. **Add files:** Create `index.html`, `style.css`, `script.js`, and `README.md`. Paste the corresponding source code and commit each file.

3. **Review:** Open the HTML file in a browser or use a local development server. Test low, medium, and high scores and checklist interactions.

4. **Optional publish:** Open repository Settings → Pages. Choose Deploy from a branch, select `main` and `/(root)`, then save.

5. **Verify:** Wait for deployment, open the Pages URL, and confirm the page loads correctly. Only submit a working URL after testing it.

6. **Submit evidence:** Provide one public repository or view-only project link, with the report and any supporting diagram/checklist in the same repository or project folder.

8. Testing Plan

Test	Expected result
Likelihood 1 × Impact 1	Score 1; Low risk.
Likelihood 3 × Impact 3	Score 9; Medium risk.
Likelihood 5 × Impact 5	Score 25; High risk.
Empty threat field	Browser prevents submission because the field is required.
Checklist items	Each checkbox can be toggled by the user.

9. Conclusion

The Threat Modeling Web Application provides a beginner-friendly way to document assets, categorize plausible threats, score risks, and plan defensive improvements. The key outcome is not just a working web page: it is a clear, prioritized security plan supported by a threat model, STRIDE register, and hardening checklist.

Submission checklist: [ ] Threat-model diagram [ ] STRIDE risk register [ ] Prioritized hardening checklist [ ] Public or view-only project link [ ] Tested web app